CHANDIGARH UNIVERSITY

RailPredict – Railway Passenger Demand Analytics System

ADBMS Mini Project Report

Submitted by: Daksh Kumar Rana
Programme: MCA – Data Science
Semester: I
Academic Session: 2026–27

Submitted to: _____
Faculty / Subject Code: _____

A database-driven analytics dashboard for railway passenger bookings, occupancy, cancellations and revenue.

INDEX

S. No.	Chapter	Page
1	Certificate and Declaration	3
2	Acknowledgement and Abstract	4
3	Introduction and Problem Statement	5
4	Objectives, Scope and Limitations	6
5	Tools and System Requirements	7
6	Database Design and Schema	8
7	Implementation and SQL Concepts	9
8	Dashboard Modules and Results	11
9	Testing and Validation	12
10	Conclusion and Future Scope	13
11	References	14

1. Certificate and Declaration

This is to certify that the project titled **RailPredict – Railway Passenger Demand Analytics System** has been prepared as an academic mini project for the Advanced Database Management Systems subject. Faculty certification and signatures may be added by the department.

Declaration: I declare that this project report is prepared for academic learning. The sample passenger and booking records are synthetic and do not represent actual railway passenger data.

2. Acknowledgement and Abstract

I express my gratitude to the faculty and department for their guidance. RailPredict is a database-backed analytics application that stores railway booking information and summarizes train occupancy, route performance, cancellations and revenue. The implementation uses SQLite for relational storage, SQL views and triggers for database-side logic, and Python with Streamlit for the dashboard.

3. Introduction and Problem Statement

Railway operators need structured information to understand passenger demand, seat utilization, cancellations and revenue. Raw booking records are difficult to interpret without aggregation. RailPredict organizes sample data into related tables and provides analytical views to support quick reporting. The system is an educational prototype and is not connected to live railway systems.

4. Objectives, Scope and Limitations

Objectives: design a relational schema; maintain booking records; validate seat capacity; record booking events; calculate occupancy and revenue; display analytical summaries.
Scope: sample stations, trains, routes, passengers and bookings; dashboard reporting; CSV export.
Limitations: synthetic dataset, single local SQLite database, no live booking integration, no payment processing, and no real-time train status.

5. Tools and System Requirements

Hardware: MacBook or computer with at least 4 GB RAM. **Software:** macOS, Python 3.10+, VS Code, SQLite (included with Python), Streamlit, Pandas and Plotly. Internet is required only to install Python packages. No Oracle account, database server, SQLcl or wallet is required.

6. Database Design and Schema

The database contains six main entities: **stations**, **trains**, **routes**, **passengers**, **bookings** and **booking_audit**. Primary keys uniquely identify records. Foreign keys connect bookings to passengers, trains and routes, and routes to stations. CHECK constraints validate fare, seat count, age and booking status. The design reduces duplication by storing train, passenger and station details separately.

7. Implementation and SQL Concepts

The SQL scripts create tables, insert synthetic sample records, define triggers and create analytical views. The capacity trigger prevents confirmed bookings from exceeding train capacity for a train and journey date. Audit triggers record inserts and booking-status changes. Views calculate train occupancy, route performance and

monthly revenue. Python functions load the SQL scripts and query the views for dashboard display. SQLite is not Oracle PL/SQL; Python functions are used instead of Oracle packages and stored procedures.

8. Dashboard Modules and Results

The Streamlit dashboard contains four modules: **Train Occupancy** (confirmed seats and occupancy percentage), **Route Analysis** (booking and cancellation counts), **Revenue** (revenue grouped by journey month), and **Bookings & Export** (filterable booking table and CSV download). Run the dashboard and insert screenshots of each tab here before submission.

9. Testing and Validation

The SQL setup was validated by creating the schema, loading sample records and creating all three analytical views in SQLite. The sample database contains 20 booking records. Recommended manual tests: insert a valid booking; attempt a booking that exceeds capacity; update a booking status and verify the audit table; compare dashboard totals with SQL view results. Add screenshots of your actual test outputs.

10. Conclusion and Future Scope

RailPredict demonstrates relational database design and practical analytics using SQL, SQLite and Python. It converts booking records into useful summaries of occupancy, route activity and revenue. Future improvements may include larger datasets, date filters, predictive demand forecasting, role-based access and integration with an authorized live data source.

11. References

1. SQLite Documentation – https://www.sqlite.org/docs.html
2. Python Documentation – https://docs.python.org/3/
3. Streamlit Documentation – https://docs.streamlit.io/
4. Pandas Documentation – https://pandas.pydata.org/docs/
5. Plotly Python Documentation – https://plotly.com/python/